

Robicsek Bence
WLEASJ
NJE
Digitális Technika II.
Ultrahangos lokátor

Specifikáció:

A projektet a BME mérnökinformatikus szakon a Témalabor tárgy beadandójaként készítettem el egy előre összeállított fizikai eszközön, ahol az Arduino lapra egy a tanszék által elkészített egyedi híd volt szerelve 4 gombbal, illetve erre csatlakozott a szervomotor, melyre mintegy forgófejként volt rászerve az ultrahangos érzékelő. Mivel az eszközt a tárgy teljesítéséig csak kölcsön kaptam, így fizikailag nem tudom bemutatni, ezért TinkerCadban létrehoztam az áramköri rajzát.

A lokátor a kezelőfelületen végrehajtott gombnyomás után 150 fokos szögben szkenneli környezetét. Az ultrahangos távolságmérő adatait, a távolság-elfordulási szög függvényértékeket egy polár koordináta rendszerben ábrázolja a LabView frontpanelen. A szkennelés után a lokátor feje a legközelebbi objektum felé fordul.

Összetevők:

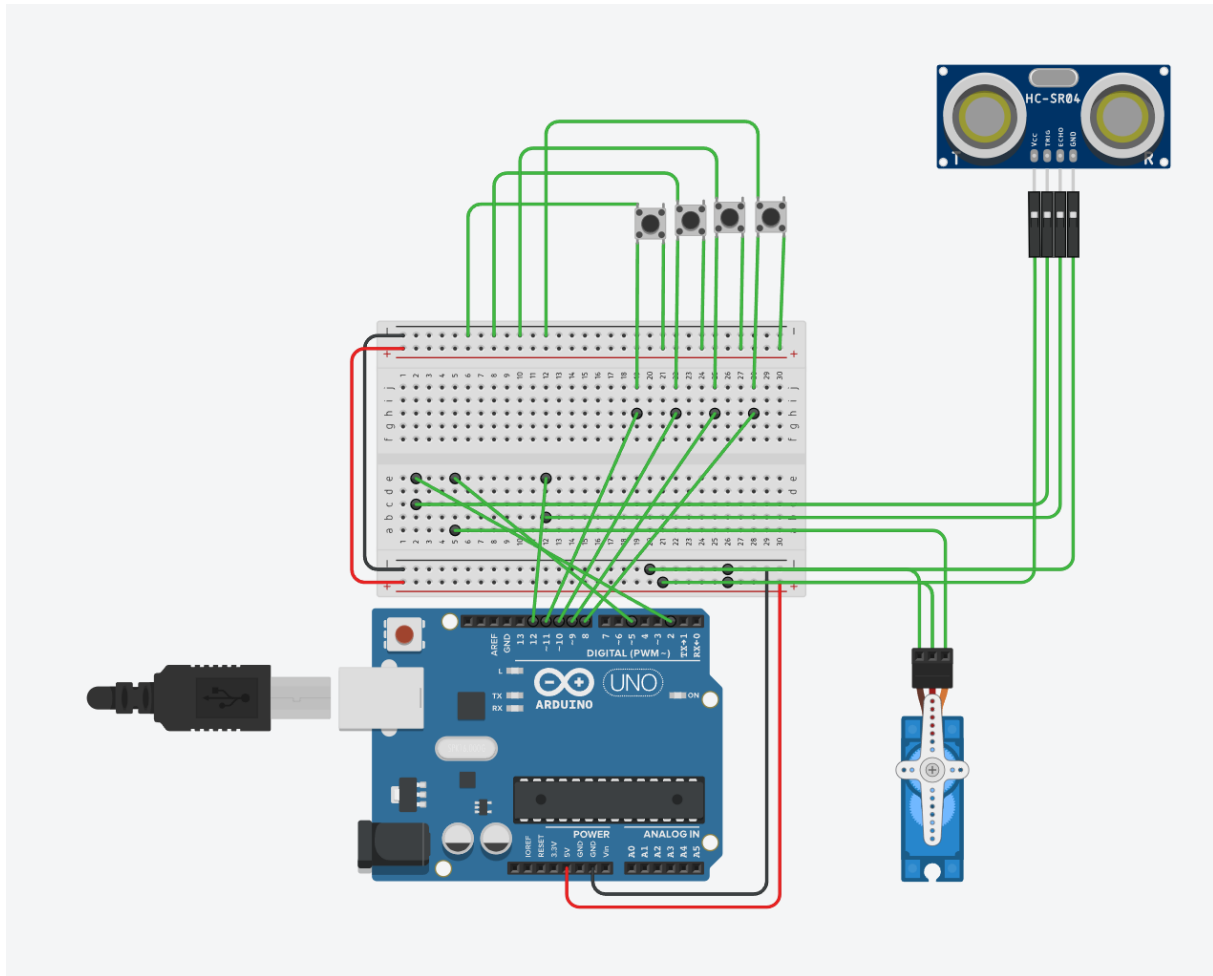
Név	Mennyiség	Összetevő
SpinBtnSweep SpinBtnClosest SpinBtnLeft SpinBtnRight	4	Nyomógomb
UArduino	1	Arduino Uno R3
DISTHC-SR04	1	Ultrahangos távolságérzékelő (4 tűs)
SERVO1	1	Pozicionális Mikroszervo

Tinkercad terv:

[Tinkercad link](#)

Megvalósítás:

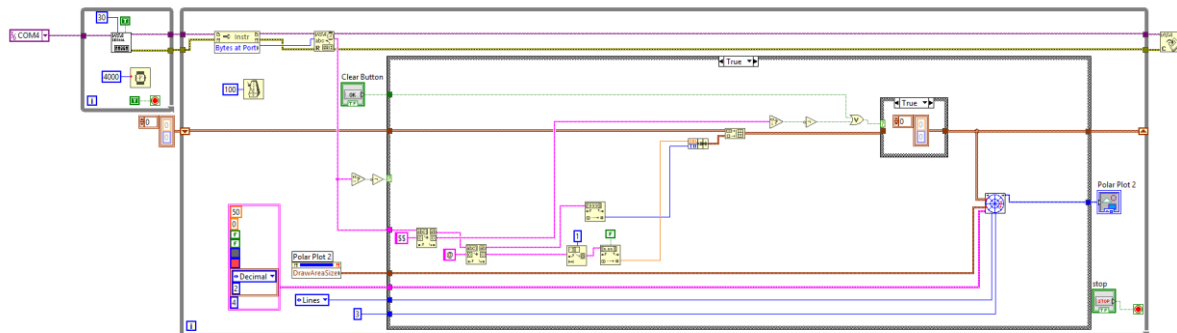
A programot az Arduino egységen valósítottam meg.



Az Ultrahangos lokátor áramköri rajza. Az ultrahangos távolságmérő fizikailag a szervomotorra van szerelve, az forgatja körbe a radart.

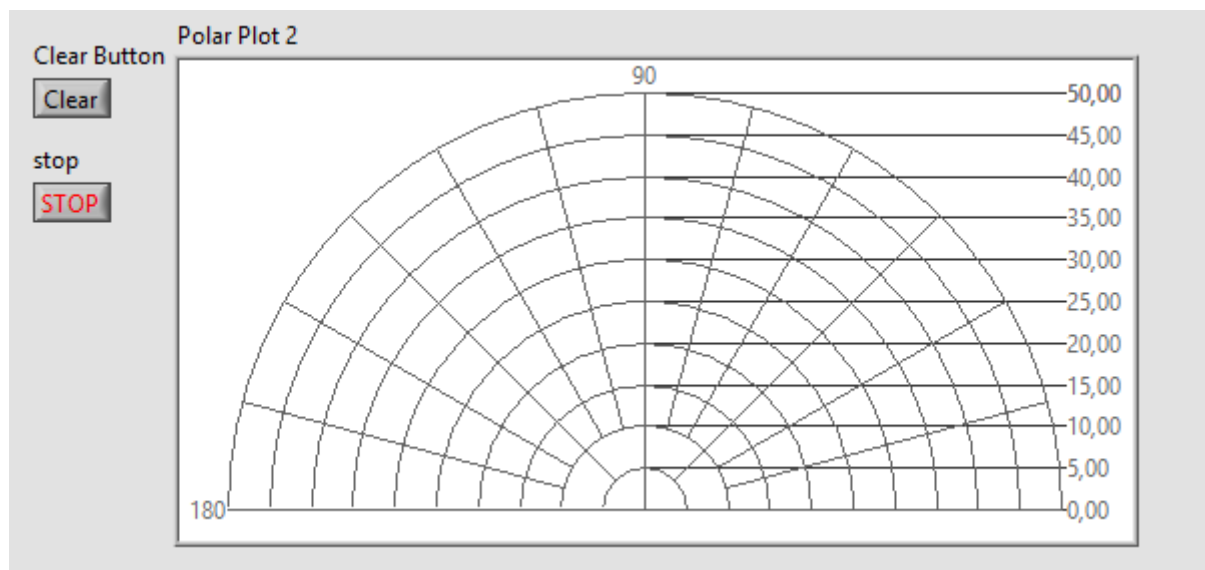
A 1-es számú gomb megnyomására a szervomotor 0-ról 180 fokra fordul, miközben az ultrahangos érzékelő kiküldi, majd olvassa a jeleket. A szervot a Servo.h könyvtár segítségével mozgatom, egy 0-180-ig iteráló for ciklus minden lépésében az iterátorral paraméterként meghívom a Servo.h könyvtár write() függvényét, mellyel az adott pozícióba állítja a szervot. A pozícióba állás után megbizonyosodok az ultrahangos érzékelő adójának kikapcsolt állapotáról, majd 10 mikró szekundumig bekapcsolom és küldetem a jelet folyamatosan, az idő lejártá után kikapcsolom és a pulseIn() függvény segítségével elmentem a jel visszaérkezésének idejét. Ezt az időt átalakítom centiméterbe a hangsebesség segítségével és megfelezem, mert ez tartalmazza az oda- és visszautat is, majd a servo pozíciót és a távolságot kiíratom a soros portra. Az adott pozícióban lévő tárgy távolságát és pozícióját elmentem egy változóba, ha eddig az volt a legközelebbi tárgy, majd a pásztázás végén a szervot ebbe a pozícióba állítom. A 2-as számú gomb megnyomására a servo rögtön a legközelebbi tárgyra ugrik. Ilyenkor a távolságot és a pozíciót nem iratom ki a soros portra. A 3-as és 4-es gombok lenyomására a servo balra, illetve jobbra fordul el, 0

és 180 fok között, és folyamatosan számítja a távolságot és a legközelebbi pozíciót, illetve írja az adatokat a soros portra.



A LabView blokk diagramja. Itt történik az ultrahangos távolságmérő jeleinek feldolgozása és kiírása.

Az adatok beolvasását és megjelenítését LabView-ban hoztam létre. A soros portot egy VISA Configure Serial Port-tal nyitom meg és várok 4 másodpercet, hogy a bootloader-üzemmód biztosan kikapcsoljon. Ennek lejárta után átlép a futási ciklusba, ahol a soros porton érkező adatokat kiolvasa, feldolgozza és megjeleníti egy polar ploton. A kezelő felületen elhelyeztem egy Clear gombot, melyet megnyomva a polar plotról letöröl minden adatot. A program a Stop gomb megnyomásáig fut, ami után a VISA Close blokkal a soros kapcsolatot lezárom. A program futása alatt a VISA Read blokk beolvasa a soros porton érkező adatokat. Ha a beérkező string a „\$\$”, akkor a polar plotról letöröl minden adatot, ez a pásztázás elindításakor történik meg. A beérkező sztringet az elválasztó karakter mentén szétbontom és átalakítom, a távolságot lebegőpontos, a pozíciót egész számértékké, mert a szervót is csak egész fokokra állítom. Ezeket egy clusterré egyesítem és hozzáadom a tömbhöz, melyet a polar plot jelenít meg. A polar plot adatainak törlése során is ennek a tömbnek elemeit törölöm ki.



A LabView frontpanel nézete a Clear és Stop gombokkal. A Polar Plot kijelzőn piros egybefüggő vonallal jelöli a lokátor körül észlelt tárgyak távolságát.

Arduino szoftver forráskódja:

```
#include <Servo.h>

#define pinTrig 2
#define pinAns 12
#define pinBtnSweep 11
#define pinBtnClosest 10
#define pinBtnLeft 9
#define pinBtnRight 8
#define pinServo 5

Servo myServo;

int posServo = 0;    // Szervó pozíciója
int posClosest = 0;  // Legközelebbi tárgy pozíciója
float distance = 0;   // Tárgy távolsága
float closestDist = 450; // Legközelebbi tárgy távolsága, kezdőérték a leírás szerinti
                        // legtávolabbi érték, 450 cm

void setup() {
  pinMode(pinBtnSweep, INPUT_PULLUP); // Pásztázó gomb
  pinMode(pinBtnLeft, INPUT_PULLUP);  // Manuális balra
  pinMode(pinBtnRight, INPUT_PULLUP); // Manuális jobbra
  pinMode(pinBtnClosest, INPUT_PULLUP); // Legközelebbi tárgyra ugró gomb
  pinMode(pinTrig, OUTPUT);
  pinMode(pinAns, INPUT);
  Serial.begin(9600);
  myServo.attach(pinServo);
  myServo.write(0);
}

void loop() {
```

```

static bool btnPrevSweep = true;
static bool btnPrevClosest = true;
bool btnStateSweep = digitalRead(pinBtnSweep);
bool btnStateClosest = digitalRead(pinBtnClosest);

if(!btnStateSweep & btnPrevSweep)
{
    Serial.println("$$");
    closestDist = 450;

    for (posServo = 0; posServo <= 180; posServo += 1) // Egyik irányba forog, 150 fok a
    maximális szöge, de 180 fokon kell megjeleníteni
    {
        myServo.write(posServo);
        delay(25);
        calculateAndSend(); // Az ultrahangos érzékelő bekapcsolása, adatok feldolgozása,
        cm-be alakítása és elküldése
        isClosest(); // A legközelebbi pozíció ellenőrzése és elmentése
        delay(25);
    }

    myServo.write(posClosest);
    posServo = posClosest;
    delay(15);
}

btnPrevSweep = btnStateSweep;

if(digitalRead(pinBtnLeft) == LOW & posServo < 180) //Manuális forgatás
balra      !!!!!150 fokig tud csak fordulni!
{
    posServo = posServo + 1;
    myServo.write(posServo);
}

```

```

    delay(20);
    calculateAndSend();
    isClosest();
    delay(25);
}

if(digitalRead(pinBtnRight) == LOW & posServo > 0) //Manuális forgatás jobbra
{
    posServo = posServo - 1;
    myServo.write(posServo);
    delay(20);
    calculateAndSend();
    isClosest();
    delay(25);
}

if(!btnStateClosest & btnPrevClosest) // A legközelebbi tárgyra fordul
{
    myServo.write(posClosest);
    posServo = posClosest;
    delay(15);
}

btnPrevClosest = btnStateClosest;
delay(25);
}

void calculateAndSend()
{
    long duration;
    Serial.flush();           // Kiürítjük a küldő buffert
    delay(15);

```

```

digitalWrite(pinTrig, LOW);      // Megbizonyodás a szonár kikapcsolt állapotáról
delayMicroseconds(2);
digitalWrite(pinTrig, HIGH);     // A szonár bekapcsolása 10 mikroszekundumig
delayMicroseconds(10);
digitalWrite(pinTrig, LOW);      // A szonár kikapcsolása
duration = pulseIn(pinAns, HIGH); // A válasz időbeni beérkezésének beolvasása
(mikroszekundum)
distance = duration * 0.0343 / 2; // Az idő átalakítása távolságra (cm) ((IDŐ/2 *
HANGSEBESSÉG))
Serial.print(posServo);          // A szervó szögének továbbítása (a Labview felé)
Serial.print("@");
Serial.println(distance);         // A távolság továbbítása (a Labview felé)
delay(15);
}

void isClosest()
{
  if(distance < closestDist)
  {
    closestDist = distance;
    posClosest = posServo;
  }
}

```